IoT-Based Pothole and Speed-Hump Detection with On-Device Machine Learning and an AWS Serverless GIS Map

A CAPSTONE PROJECT REPORT

submitted in partial fulfilment of the requirements for the course

CLOUD COMPUTING

by

[STUDENT NAME] ([REGISTER NUMBER])

Under the guidance of

[GUIDE NAME], [DESIGNATION]

[DEPARTMENT / SCHOOL]

[INSTITUTION NAME, CITY]

October 2026

BONAFIDE CERTIFICATE

This is to certify that the capstone project report entitled “IoT-Based Pothole and Speed-Hump Detection with On-Device Machine Learning and an AWS Serverless GIS Map” is a bonafide record of the work carried out by **[STUDENT NAME] ([REGISTER NUMBER])** under my supervision, in partial fulfilment of the requirements for the course Cloud Computing. The contents of this report have not been submitted elsewhere for the award of any degree or diploma.

[GUIDE NAME]
Project Guide

[HEAD OF DEPARTMENT]
Head of the Department

Internal Examiner

External Examiner

DECLARATION

I hereby declare that the capstone project report entitled “IoT-Based Pothole and Speed-Hump Detection with On-Device Machine Learning and an AWS Serverless GIS Map” submitted for the course Cloud Computing is a record of original work done by me under the guidance of [GUIDE NAME]. The hardware, firmware, machine-learning pipeline and cloud deployment described here were designed, built and tested as part of this project. Sources of external information are acknowledged in the references.

Place: [CITY] **[STUDENT NAME]**
Date: [DATE]

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, [GUIDE NAME], for the guidance and encouragement throughout this project, and to the Head of the Department and the faculty of [DEPARTMENT] for the facilities and support provided. I also thank my friends and family for their help with testing and for their constant support.

[STUDENT NAME]

ABSTRACT

Potholes damage vehicles and cause accidents, yet road agencies typically learn about them late and through manual complaints. This project presents an end-to-end, low-cost system that detects potholes and speed humps automatically from a moving vehicle and publishes them to a shared, live map in the cloud. An ESP32 microcontroller reads a three-axis accelerometer (MPU-6050 or ADXL345) at 100 Hz and a u-blox NEO-7M GPS receiver at 5 Hz. An orientation-independent event detector tracks the gravity vector, extracts the vertical dynamic acceleration, and cuts a 1.5 s window around every jolt larger than 0.25 g. Twenty shape features computed from the window are classified on the device by a 10-tree random forest of only 17 KB, which distinguishes potholes (drop first) from speed humps (lift first) and from ordinary disturbances.

Each pothole or speed hump is sent with the GPS position captured at the instant of impact as a ~350-byte JSON message over MQTT with mutual-TLS X.509 authentication to AWS IoT Core. A fully serverless backend, described as infrastructure-as-code with AWS SAM, routes messages through an IoT rule to an AWS Lambda function that validates them, stores the raw report in Amazon DynamoDB and folds it into a geohash map cell with atomic counters, so that repeated reports of the same pothole by any number of vehicles become a single marker whose report count expresses confidence. A second Lambda behind an Amazon API Gateway HTTP API serves a Leaflet/OpenStreetMap web map and a standard GeoJSON feed usable in QGIS.

Under five-fold session-grouped cross-validation on 1 625 events from 58 sessions, the deployed model reaches a macro-F1 of 0.88 (pothole F1 0.90, speed-hump F1 0.90); on held-out real motions never used in any form it labels 91 % of drop-then-lift events as potholes and 80 % of lift-then-drop events as speed humps. The complete chain from sensor to map was deployed in the AWS ap-south-1 region and verified with the real device: all six reports produced were stored and mapped, about two seconds after the trigger, at a running cost within the AWS Free Tier.

Keywords: pothole detection, Internet of Things, edge machine learning, random forest, ESP32, MQTT, AWS IoT Core, AWS Lambda, DynamoDB, serverless computing, GIS, GeoJSON.

TABLE OF CONTENTS

ABSTRACT ..... **v**
LIST OF FIGURES ..... **viii**
LIST OF TABLES ..... **ix**
LIST OF ABBREVIATIONS ..... **x**
1 INTRODUCTION **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope 2
1.5 Organisation of the Report 2
2 LITERATURE SURVEY **3**
2.1 Vehicle-Based Road-Surface Monitoring 3
2.2 Machine Learning for Road Events 3
2.3 Edge Computing and Serverless Cloud 3
2.4 Geospatial Representation 4
2.5 Gap Addressed 4
3 SYSTEM REQUIREMENTS AND TOOLS **5**
3.1 Functional Requirements 5
3.2 Non-Functional Requirements 5
3.3 Hardware 5
3.4 Software and Services 6
4 SYSTEM DESIGN **7**
4.1 Overall Architecture 7
4.2 Edge Design 7
4.3 Message Contract 9
4.4 Cloud Design 9
4.5 Security Design 10
5 IMPLEMENTATION **11**
5.1 Firmware 11
5.2 Machine-Learning Pipeline 11
5.3 Cloud Deployment 12
5.4 Web Map 13
6 RESULTS AND DISCUSSION **14**
6.1 Feature Design 14

6.2 Model Selection....15
6.3 End-to-End Verification....17
6.4 Cost....17
6.5 Problems Encountered....18
6.6 Limitations....18
7 CONCLUSION AND FUTURE WORK....19
7.1 Conclusion....19
7.2 Future Work....19
REFERENCES....20
APPENDIX A: IOT POLICY AND IOT RULE....21
APPENDIX B: REPOSITORY STRUCTURE....21

LIST OF FIGURES

Figure 1: System architecture: edge classification and serverless cloud.....7
Figure 2: Vertical dynamic acceleration around the trigger: a real speed hump (lift first) and the mirrored pothole training example (drop first).....8
Figure 3: Macro-F1 of axis-statistics (v1) versus shape (v2) features on simulated data.....14
Figure 4: Cross-validated macro-F1 of all 17 candidate models.....15
Figure 5: Confusion matrix of forest_n10_d8 (five-fold session-grouped CV).....16
Figure 6: Ten most important features of the deployed forest.....16

LIST OF TABLES

Table 1: Hardware components.....5
Table 2: Software, libraries and cloud services.....6
Table 3: The 20 features computed on the device.....8
Table 4: AWS services and their roles.....9
Table 5: Machine-learning pipeline stages.....11
Table 6: Cross-validated per-class performance of the deployed model.....15
Table 7: Reports received from the device (coordinates omitted).....17
Table 8: Verification summary.....17
Table 9: Problems encountered and their solutions.....18

LIST OF ABBREVIATIONS

API	Application Programming Interface
AWS	Amazon Web Services
CA	Certificate Authority
CV	Cross-Validation
GIS	Geographic Information System
GNSS / GPS	Global Navigation Satellite System / Global Positioning System
HDOP	Horizontal Dilution of Precision
HTTP(S)	Hypertext Transfer Protocol (Secure)
I2C	Inter-Integrated Circuit bus
IAM	AWS Identity and Access Management
IaC	Infrastructure as Code
IoT	Internet of Things
JSON	JavaScript Object Notation
MQTT	Message Queuing Telemetry Transport
NMEA	National Marine Electronics Association (GPS sentence format)
OSM	OpenStreetMap
QoS	Quality of Service
RTOS	Real-Time Operating System
SAM	AWS Serverless Application Model
TLS	Transport Layer Security
TTL	Time To Live
UART	Universal Asynchronous Receiver-Transmitter
UBX	u-blox binary protocol

CHAPTER 1

INTRODUCTION

1.1 Background

Road-surface defects are a persistent safety and maintenance problem, particularly on roads with heavy traffic and seasonal rain. Potholes damage tyres, rims and suspensions, cause sudden braking and swerving, and are a recurring cause of two-wheeler accidents. Road agencies usually learn about potholes through manual inspection or citizen complaints, which is slow, incomplete and difficult to prioritise.

Every vehicle that drives over a pothole experiences a characteristic vertical acceleration. Low-cost MEMS accelerometers, GPS receivers and Wi-Fi-capable microcontrollers make it possible to turn ordinary vehicles into mobile road sensors, and managed cloud IoT services make it possible to aggregate their reports into a shared map without operating any servers. This project combines both: machine learning at the edge to recognise road events, and a serverless cloud backend to collect, aggregate and visualise them.

1.2 Problem Statement

Design and implement a low-cost system that (i) detects potholes automatically while driving, (ii) distinguishes potholes from speed humps, which produce similarly large vertical jolts, (iii) attaches an accurate geographic position to every detection, and (iv) publishes detections from any number of vehicles to a cloud backend that aggregates repeated reports into a live, shareable map.

1.3 Objectives

1. Build a vehicle-mounted device from an ESP32, a three-axis accelerometer and a GPS receiver.
2. Develop an event detector that works at any mounting angle and ignores flat road.
3. Train a compact classifier that runs on the microcontroller and separates pothole, speed hump and normal events.

4. Publish classified events with GPS position securely over MQTT/TLS to AWS IoT Core.
5. Deploy a serverless backend (IoT rule, Lambda, DynamoDB, API Gateway) as infrastructure-as-code.
6. Aggregate reports from many vehicles into map markers and serve them as a web map and a GeoJSON feed.
7. Verify the complete chain end-to-end with the real device.

1.4 Scope

The project covers the hardware prototype, three ESP-IDF firmware applications (data collector, bench inference, IoT publisher), a Python machine-learning pipeline (collection, simulation, augmentation, training, C export) and an AWS backend with a web map. Model training uses a car-suspension simulator and hand-made bench recordings; collecting and validating on real road data is identified as future work.

1.5 Organisation of the Report

Chapter 2 reviews related work. Chapter 3 lists the requirements and tools. Chapter 4 presents the system design and Chapter 5 its implementation on the device, in the machine-learning pipeline and in the cloud. Chapter 6 reports the results and discusses problems encountered, and Chapter 7 concludes with future work.

CHAPTER 2

LITERATURE SURVEY

2.1 Vehicle-Based Road-Surface Monitoring

Eriksson et al. introduced *Pothole Patrol* [1], in which taxis equipped with accelerometers and GPS detected potholes using a cascade of hand-tuned filters (speed, high-pass, z-peak, x-z ratio, speed-versus-z ratio) and clustered detections across vehicles to suppress false positives. Their key observation, that repeated reports at the same location increase confidence, is used directly in the aggregation of this project.

Mohan et al. presented *Nericell* [2], which used smartphone accelerometers, microphones and GPS to monitor braking, bumps and honking, and introduced virtual re-orientation of an arbitrarily placed accelerometer using the gravity vector. Mednis et al. [3] compared threshold-based algorithms (Z-THRESH, Z-DIFF, STDEV(Z), G-ZERO) for real-time pothole detection on Android phones and reported true-positive rates of up to about 90 %, while noting the sensitivity of fixed thresholds to vehicle and mounting.

2.2 Machine Learning for Road Events

Later work replaced hand-tuned thresholds by supervised classifiers on windowed features. Tree ensembles such as random forests [4] and extremely randomised trees [5] are attractive for embedded deployment: they need no feature scaling, handle heterogeneous features, provide feature importances and compile to a few kilobytes of comparisons. This project uses scikit-learn [6] to train and select among such models and exports the chosen forest to C.

2.3 Edge Computing and Serverless Cloud

Edge computing moves computation close to where data is produced to reduce bandwidth and latency [7]. Classifying road events on the device reduces the uplink from about 1 KB/s of raw samples per vehicle to a few hundred bytes per event. On the cloud side, serverless computing [8] (functions as a service and managed databases billed per request) suits the bursty, low-volume traffic of a road-monitoring fleet. MQTT [9] is the dominant lightweight publish/subscribe protocol for IoT

devices, and AWS IoT Core [10] provides a managed MQTT broker with per-device X.509 authentication and a rules engine.

2.4 Geospatial Representation

GeoJSON [11] is the standard open format for exchanging geographic features over the web and is read by desktop GIS (QGIS) and web libraries such as Leaflet [12]. Geohash [13] encodes latitude and longitude into a string whose prefix identifies a rectangular cell, which makes it a natural partition key for spatial aggregation in a key-value store.

2.5 Gap Addressed

Earlier systems either used fixed thresholds that depend on vehicle and mounting, ran on smartphones, or stored data on self-managed servers. This project combines an orientation-independent detector, a small learned classifier running on a microcontroller, and a fully managed serverless backend with crowd aggregation and open GIS output, all deployable with a single infrastructure-as-code template.

CHAPTER 3

SYSTEM REQUIREMENTS AND TOOLS

3.1 Functional Requirements

- Detect vertical jolts while driving and classify each as pothole, speed hump or normal.
- Record the GPS position at the moment of impact.
- Publish potholes and speed humps (not normal events) with position, confidence and severity.
- Keep working without network coverage and send queued events after reconnection.
- Store every report; show aggregated potholes and humps on a public web map and as GeoJSON.
- Filter the map by event type and minimum number of reports.

3.2 Non-Functional Requirements

- Low cost and low power: a single microcontroller with no external compute.
- Security: mutual-TLS device authentication; devices may only publish to their topic.
- Scalability without servers to manage; costs proportional to usage.
- Reproducibility: the entire backend described as code and deployable with one command.

3.3 Hardware

Table 1: Hardware components

Component	Specification	Interface / pins
ESP32 DevKit V1 (ESP32-WROOM-32)	2 × Xtensa LX6 240 MHz, 520 KB RAM, Wi-Fi 2.4 GHz	USB power
MPU-6050 (GY-521) — or ADXL345 (GY-291)	3-axis accelerometer, ±8 g, 100 Hz, auto-detected	I2C: SDA GPIO21, SCL GPIO22
u-blox NEO-7M (GY-NEO7M)	GPS, configured to 5 Hz, GGA + RMC	UART2: RX GPIO16, TX GPIO17
On-board LED	GPS / publish status	GPIO2
Phone hotspot	2.4 GHz Wi-Fi uplink	—

3.4 Software and Services

Table 2: Software, libraries and cloud services

Area	Tools
Firmware	ESP-IDF v6.1, FreeRTOS, esp-mqtt 1.1, mbedTLS, xtensa-esp-elf GCC 15.2, esptool
Machine learning	Python 3, NumPy, SciPy, pandas, scikit-learn, pyserial, matplotlib, pytest, ctypes
Cloud	AWS IoT Core, AWS Lambda (Python 3.12, arm64), Amazon DynamoDB, Amazon API Gateway (HTTP API), AWS CloudFormation, Amazon S3, Amazon CloudWatch Logs, AWS IAM
Cloud tooling	AWS CLI, AWS SAM CLI 1.169, boto3
GIS	Leaflet 1.9, OpenStreetMap tiles, GeoJSON, geohash; QGIS / kepler.gl compatible
Engineering	Git and GitHub, Markdown and Mermaid documentation

CHAPTER 4

SYSTEM DESIGN

4.1 Overall Architecture

Figure 1 shows the architecture. The edge device performs sensing, detection and classification and sends only classified events. The cloud side consists exclusively of managed AWS services in the ap-south-1 (Mumbai) region: IoT Core receives MQTT messages, an IoT rule forwards them to an ingest Lambda function that writes to DynamoDB, and an HTTP API with a second Lambda function serves the map and the GeoJSON feed.

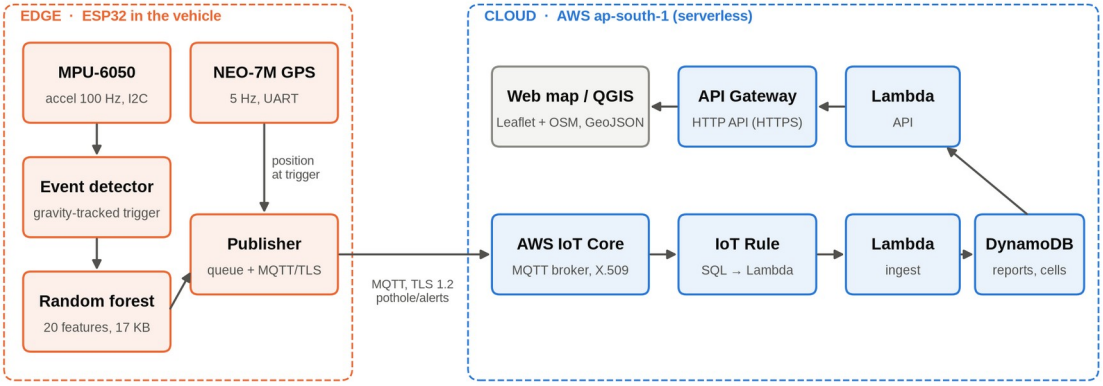


Figure 1: System architecture: edge classification and serverless cloud

4.2 Edge Design

4.2.1 Orientation-independent event detection

Every accelerometer sample **a** (in g) updates a slow low-pass estimate of gravity **g** (time constant about 1 s, adapting ten times more slowly during an event). The vertical dynamic acceleration is $v = a \cdot \hat{g} - |g|$ and the horizontal component is $h = |a - g - v\,\hat{g}|$. An event is triggered when |v| exceeds 0.25 g; the window contains 0.5 s before and 1.0 s after the trigger (150 samples at 100 Hz), which covers the run-up and both axles at normal speeds, followed by a 0.3 s cooldown. Flat road never triggers and is therefore never classified, which suppresses false positives and keeps the CPU almost idle.

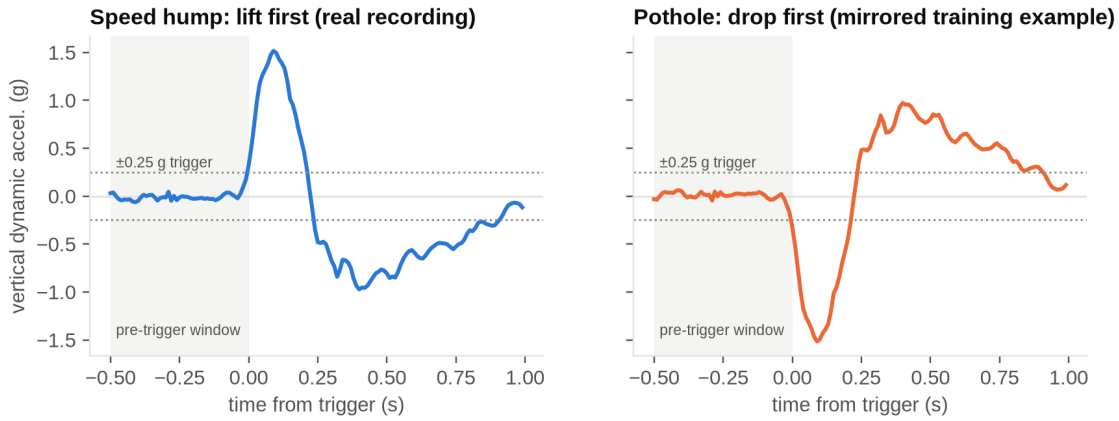


Figure 2: Vertical dynamic acceleration around the trigger: a real speed hump (lift first) and the mirrored pothole training example (drop first)

4.2.2 Features

Twenty features summarise each window (Table 3). Unlike per-axis statistics, they are independent of the mounting angle and capture the *order* and *shape* of the excursion, which is what separates a pothole (the wheel first drops, then hits the far edge) from a speed hump (the wheel is first lifted).

Table 3: The 20 features computed on the device

#	Features	Purpose
0–4	v_max, v_min, v_p2p, v_std, v_mean	size of the event
5	first_peak	sign of the first large excursion (drop first = pothole)
6	t_max_minus_t_min	order of the extremes
7	active_time	duration above threshold
8–10	jerk_max, hf_ratio, crest	sharp edges, high-frequency energy, spikiness
11–13	h_max, h_std, h_v_ratio	roll: single-wheel potholes vs. full-width humps
14	zcr	oscillation rate
15	pre_rms	roughness of the road just before the event
16–17	neg_area, pos_area	amount of downward vs. upward motion
18–19	skew, kurtosis	asymmetry and peakedness

4.2.3 Firmware tasks

The firmware runs four FreeRTOS tasks: a high-priority sensor task pinned to core 1 that reads the accelerometer, runs the detector and classifier and enqueues alerts; a publish task that sends queued alerts when MQTT is connected (an alert leaves the 32-entry queue only after a successful publish); an LED/status task; and the GPS task that parses NMEA sentences in the background. The GPS state is copied at the trigger instant, so the reported position is where the wheel hit, not where the car was after the 1 s window.

4.3 Message Contract

Potholes and speed humps with a class probability of at least 0.6 are published to topic `pothole/alerts` with QoS 1. Without a fresh GPS fix (older than 3 s) the coordinates are null.

```
{"device_id":"pothole-
6acecc","event":"POTHOLE","confidence":0.75,"severity_g":0.98,
 "coordinates":[<lat>,<lon>],"lat":<lat>,"lon":<lon>,
 "gps":{"fix":true,"sats":7,"hdop":2.4,"speed_kmh":0.5,"age_ms":1167},
 "utc":"2026-10-21T21:38:06.800Z","uptime_ms":74668,"model":"forest_n10_d8"}
```

4.4 Cloud Design

Table 4: AWS services and their roles

Service	Resource	Role
AWS IoT Core	Thing, X.509 certificate, policy	device identity, mutual-TLS authentication, publish-only authorisation
AWS IoT Core	Topic rule	SELECT *, timestamp() AS received_ms, clientid() AS clientid FROM 'pothole/alerts' → Lambda
AWS Lambda	IngestFunction	validate, store raw report, aggregate into geohash cell
AWS Lambda	ApiFunction	serve the HTML map and the GeoJSON feed
Amazon DynamoDB	PotholeTable (on-demand, TTL)	raw reports (90 days) and aggregated map cells
Amazon API Gateway	HTTP API, $default stage	GET / and GET /potholes over HTTPS
AWS IAM	two execution roles	CRUD for ingest, read-only for the API
AWS CloudFormation / SAM	stack pothole-map	deploy, update and delete all of the above as one unit
Amazon S3, CloudWatch Logs	SAM artefact bucket, log groups	deployment packages; one log line per message

4.4.1 Data model

A single DynamoDB table holds two item types. Raw reports use the partition key `REPORT#<device_id>` and the receive time as sort key and expire after 90 days via TTL. Map cells use `CELL#<geohash-8>` and `TYPE#POTHOLE` or `TYPE#SPEED_HUMP`; they hold running sums of latitude, longitude and confidence, a report count, the set of reporting devices, the maximum severity and first/last-seen times. Updates use atomic `ADD` expressions, so concurrent reports from many vehicles need no read-modify-write cycle.

4.4.2 Aggregation and validation

A report is mapped only if its event is POTHOLE or SPEED_HUMP, its coordinates are present, valid and not (0, 0), and its HDOP is at most 5; otherwise it is

stored with the reason it was skipped. A geohash-8 cell measures about 38 m × 19 m. At query time the API merges cells of the same type whose centroids lie within 15 m (a pothole on a cell border), weighting by report count, and returns a GeoJSON FeatureCollection with the number of reports and devices, mean confidence, maximum severity and timestamps of each marker.

4.5 Security Design

- Each device has its own X.509 certificate; the private key never leaves the device and is git-ignored.
- The IoT policy only allows iot:Connect and iot:Publish to topic pothole/alerts; the device cannot subscribe or read.
- The ingest Lambda has no public endpoint; only the IoT rule can invoke it.
- Lambda roles follow least privilege (read/write for ingest, read-only for the API).
- The map API is public and read-only; it exposes positions and timestamps, not credentials.

CHAPTER 5

IMPLEMENTATION

5.1 Firmware

Three ESP-IDF projects share drivers in a common `components/` directory: `accel` (I2C driver that auto-detects an ADXL345 or MPU-6050 and delivers sample-exact 100 Hz data; the MPU-6050 is polled on its DATA_RDY flag because its byte FIFO was found to desynchronise), `neo7m_gps` (UART reader, NMEA RMC/GGA parser with checksum, UBX configuration to 5 Hz), `road_event` (detector and features) and `pothole_model` (the generated classifier header). `esp32_collector` streams raw samples for training, `esp32_inference` prints every classified event for bench tests, and `esp32_iot` is the deployed application. The application image is about 0.95 MB, so the single-app-large partition table is used.

The `road_event` C source is pure C99 without ESP-IDF dependencies. The same file is compiled into a shared library on the development computer and called from Python through ctypes, so the features used for training are bit-for-bit those computed on the device.

5.2 Machine-Learning Pipeline

Table 5: Machine-learning pipeline stages

Script	Output	Function
ml/collect.py	data/<label>/*.csv*, .marks	record labelled sessions over serial; guided mode prompts each repetition
ml/simulate.py	data/synthetic	quarter-car model: random car, speed, mounting angle, ISO 8608 roughness [14], joints, manholes
ml/augment.py	data/bench_aug	rotation, amplitude 0.7–1.35×, time stretch 0.85–1.2×, fresh noise
ml/dataset.py	events + features	runs the firmware's C detector via ctypes; drops implausible data
ml/train.py	pothole_model.h, model/report.md	model search, session-grouped CV, C export and check

Data. The deployed model was trained on 1 625 events from 58 sessions (normal 398, speed hump 660, pothole 567): simulated drives, augmented bench recordings and the real bench recordings themselves. The real data are hand motions with the MPU-6050 mounted upright (two normal sessions with 15 events and two

speed-hump sessions with 87 clean lift-first events after 42 failed attempts were discarded). Because no clean real pothole was recorded, pothole examples are the real humps mirrored vertically plus parametric drop–jolt–ringing events. Before augmentation, real events are filtered by their defining shape.

Model selection. Seventeen candidates were evaluated: decision trees (depth 3–8), random forests and extremely randomised trees (10 or 25 trees, depth 4–8), all with balanced class weights. Each was scored by macro-F1 under five-fold *session-grouped* cross-validation, so that no recording session appears in both training and test folds. The smallest model within 0.01 of the best score is selected.

Export. The forest is written as flat node arrays (feature index, threshold, children) in flash, with leaf class probabilities quantised to uint8. The exporter verifies that the generated C code reproduces scikit-learn's predictions on every training event.

5.3 Cloud Deployment

5.3.1 Device provisioning

A script, `cloud/provision_device.sh`, registers the device in the AWS account with the AWS CLI: it creates the IoT thing `pothole-esp32` and the policy `PotholeDevicePolicy`, generates an active key pair and certificate, attaches policy and thing to the certificate, downloads Amazon Root CA 1, and writes the account's ATS data endpoint into the firmware's git-ignored `secrets.h`. The firmware is then rebuilt and flashed; no firmware source changes are needed.

```
cloud/provision_device.sh ap-south-1 pothole-esp32
cd esp32_iot && idf.py build && idf.py -p /dev/ttyUSB0 flash monitor
```

5.3.2 Infrastructure as code

The backend is a single AWS SAM template (`cloud/template.yaml`) defining the table, both functions with their IAM policies, the IoT rule and the HTTP API. The first deployment was created as a CloudFormation change set, reviewed (eleven resources, all additions) and executed; updates are a single command.

```
cd cloud
sam validate --lint && sam build
sam deploy --stack-name pothole-map --region ap-south-1 --resolve-s3 --
capabilities CAPABILITY_IAM
```

5.3.3 Lambda functions

Both functions are in `cloud/src/potholes.py` (Python 3.12, boto3 only). `ingest_handler` validates the message, writes the raw report, computes the geohash and performs one atomic `UpdateItem` with `ADD` for the counters and sums and a conditional update for the maximum severity. `api_handler` returns the map page for `GET /` and, for `GET /potholes`, scans the cell items, merges neighbours and returns GeoJSON, with optional `type` and `min_reports` filters.

5.4 Web Map

The map page uses Leaflet with OpenStreetMap tiles and needs no API key. It polls the GeoJSON feed every 30 s, draws circle markers whose radius grows with the report count (red for potholes, blue for speed humps), shows report count, devices, confidence, severity and timestamps in a popup with a Google Maps link, and offers filters by type and minimum report count. Potholes are drawn after speed humps so that nearby hump markers never hide them.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Feature Design

On simulated drives with random mounting angles, the original pipeline that used per-axis minimum, maximum and mean values reached a macro-F1 of only 0.20 with a decision tree and 0.28 with a random forest, because axis statistics mostly encode the mounting. The orientation-independent shape features raise this to 0.84 and 0.92 respectively (Figure 3).

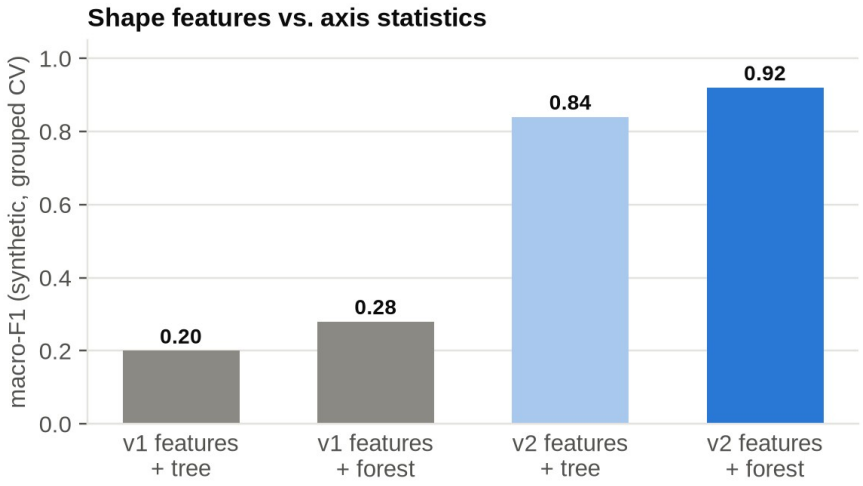


Figure 3: Macro-F1 of axis-statistics (v1) versus shape (v2) features on simulated data

6.2 Model Selection

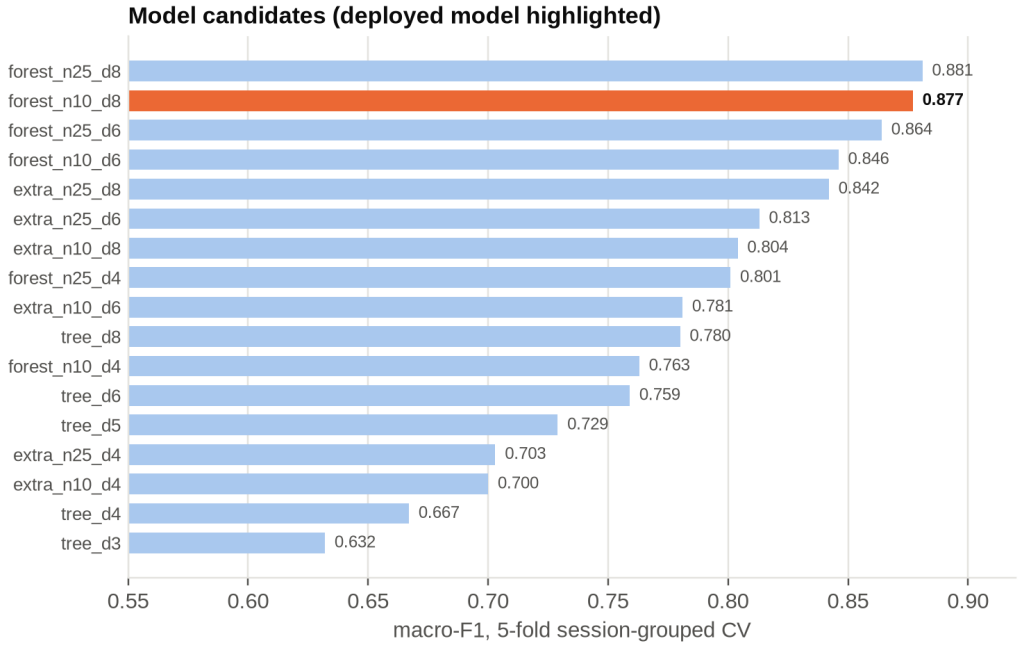


Figure 4: Cross-validated macro-F1 of all 17 candidate models

The 25-tree depth-8 forest scored highest (0.881) but needs 3 769 nodes. The 10-tree depth-8 forest scored 0.877 with 1 452 nodes (about 17 KB of flash) and was selected; its C export agrees with scikit-learn on 100 % of events and classifies a window in microseconds.

Table 6: Cross-validated per-class performance of the deployed model

Class	Precision	Recall	F1-score	Support
normal	0.83	0.84	0.83	398
speed hump	0.91	0.89	0.90	660
pothole	0.89	0.91	0.90	567
macro average	0.88	0.88	0.88	1 625

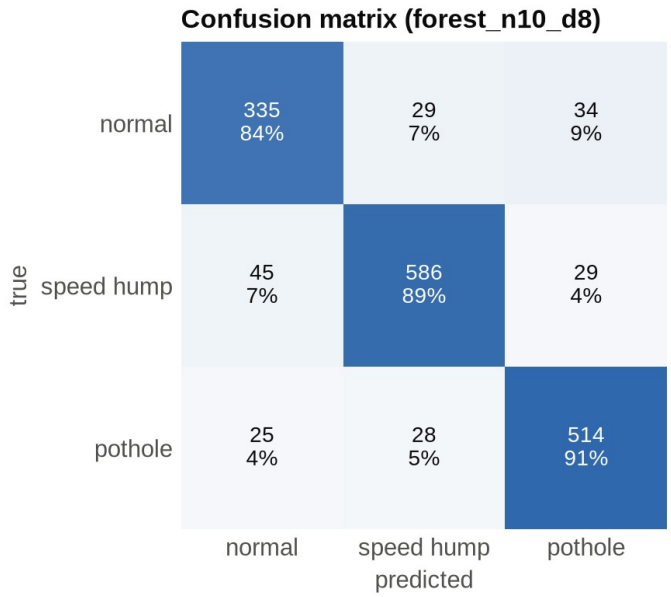


Figure 5: Confusion matrix of forest_n10_d8 (five-fold session-grouped CV)

The most important features are `t_max_minus_t_min` (0.187) and `first_peak` (0.130), the order of the extremes and the sign of the first excursion, followed by `skew` and `hf_ratio` (Figure 6). This confirms the physical intuition that potholes and humps differ mainly in which direction the wheel moves first.

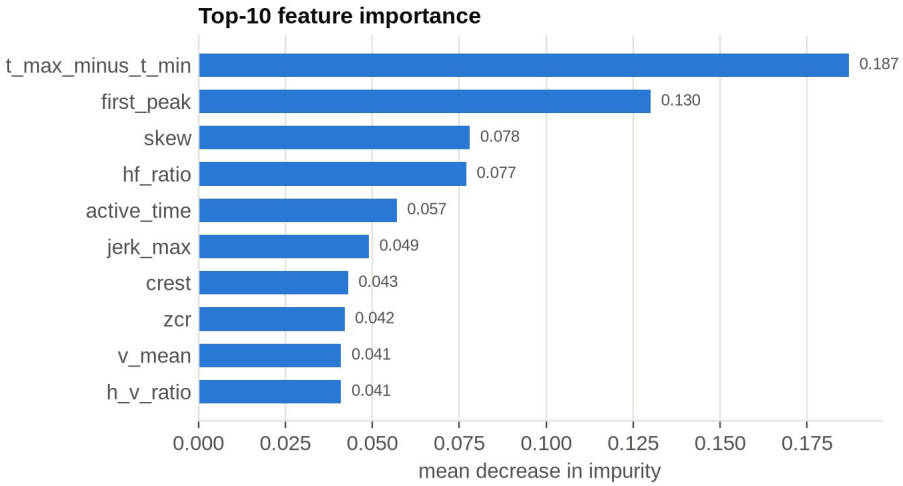


Figure 6: Ten most important features of the deployed forest

A leak-free check augmented from only half of the real humps and tested on the other half, which was never seen in any form: 80 % of the unseen lift-then-drop motions were classified as speed humps and 91 % of the same motions mirrored were classified as potholes. On the device, pothole, speed-hump and normal desk motions are all recognised, and no false events occur at rest.

6.3 End-to-End Verification

The backend was deployed to ap-south-1 and the device was provisioned and flashed. With the device on a phone hotspot and a GPS fix of 7–8 satellites, the serial log showed `MQTT connected to AWS IoT` within 14 s of boot. Table 7 lists the reports received from the device; all were stored and mapped.

Table 7: Reports received from the device (coordinates omitted)

UTC (2026-10-21)	Event	Confidence	Severity	Satellites / HDOP
21:34:14	SPEED_HUMP	0.72	0.79 g	7 / 2.4
21:37:43	SPEED_HUMP	0.78	0.89 g	8 / 0.9
21:38:06	POTHOLE	0.75	0.98 g	7 / 2.4
21:40:07	SPEED_HUMP	0.71	0.99 g	fix
21:40:34	POTHOLE	0.73	0.66 g	fix
21:41:09	POTHOLE	0.64	0.80 g	fix

The three pothole reports, a few metres apart, were merged by the API into a single marker with `reports: 3`. The cloud path was also tested independently by publishing a synthetic message with the AWS CLI, and the Lambda logic by an offline unit test with an in-memory table. The latency from trigger to database is about 1.5–2 s, dominated by the 1 s post-trigger window.

Table 8: Verification summary

Test	Method	Result
Lambda logic	offline unit test, in-memory table	passed
Template	sam validate --lint	valid
Cloud path	synthetic publish via aws iot-data publish	marker appeared in GeoJSON
Map page	HTTP GET	200 OK
Full system	real device, real GPS fix	6 / 6 reports mapped; 3 potholes → 1 marker

6.4 Cost

For a handful of devices all services stay within the AWS Free Tier: IoT Core messaging, Lambda invocations, DynamoDB on-demand requests, HTTP API requests, a few kilobytes in S3 and CloudWatch Logs. The expected monthly cost of the prototype is effectively zero, and costs grow linearly with the number of events rather than with provisioned capacity.

6.5 Problems Encountered

Table 9: Problems encountered and their solutions

Problem	Cause	Solution
v1 labels did not match the signal	recorder read stale serial data; windows not aligned to the event	event-aligned windows; guided collection
Sessions of random ±8 g values	MPU-6050 byte FIFO desynchronised	read each sample on DATA_RDY
False speed humps at rest	MPU-6050 returns 0 g as first sample after reset	fast gravity convergence during warm-up
Pothole never predicted	35 % of hump labels were failed drop-first attempts	shape filtering; potholes from mirrored humps
Device could not reach AWS	firmware still had another account's endpoint, certificate and Wi-Fi	provision_device.sh, new secrets.h, reflash
Potholes appeared missing on the map	a nearby hump marker was drawn on top at the default zoom	draw potholes last; allow closer auto-zoom

6.6 Limitations

- The model has seen simulated car physics and hand motions only; real-road accuracy remains to be measured.
- No clean real pothole recordings exist; pothole examples are mirrored humps and parametric events.
- GPS error of a few metres is comparable to pothole size; the 15 m merge radius may join very close potholes.
- The public map has no authentication and no way to mark potholes as repaired.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

The project delivers a complete, working pipeline from an accelerometer in a vehicle to a pin on a shared web map. An orientation-independent detector and twenty shape features allow a 17 KB random forest on an ESP32 to separate potholes from speed humps with a cross-validated macro-F1 of 0.88. Events are published securely over MQTT/TLS to AWS IoT Core, and a serverless backend defined in one SAM template stores, aggregates and serves them as a Leaflet map and as open GeoJSON. The deployment was verified end-to-end with the real device at negligible cost, showing that edge machine learning combined with managed cloud services is a practical basis for crowd-sourced road monitoring.

7.2 Future Work

- Record guided and free drives in real vehicles and retrain on road data; evaluate per-speed accuracy.
- Add an authenticated endpoint to mark potholes as fixed and age out old reports.
- Use AWS IoT device shadows and OTA updates to distribute retrained models to the fleet.
- Evaluate a small 1-D convolutional network (TensorFlow Lite Micro with ESP-NN) once several hundred real events per class exist.
- Replace the bootstrap root access key by an IAM user and narrow the IoT policy to the device's client ID.

REFERENCES

[1] J. Eriksson, L. Girod, B. Hull, R. Newton, S. Madden, and H. Balakrishnan, “The Pothole Patrol: Using a mobile sensor network for road surface monitoring,” in *Proc. 6th Int. Conf. Mobile Systems, Applications, and Services (MobiSys)*, 2008, pp. 29–39.

[2] P. Mohan, V. N. Padmanabhan, and R. Ramjee, “Nericell: Rich monitoring of road and traffic conditions using mobile smartphones,” in *Proc. 6th ACM Conf. Embedded Network Sensor Systems (SenSys)*, 2008, pp. 323–336.

[3] A. Mednis, G. Strazdins, R. Zviedris, G. Kanonirs, and L. Selavo, “Real time pothole detection using Android smartphones with accelerometers,” in *Proc. Int. Conf. Distributed Computing in Sensor Systems and Workshops (DCOSS)*, 2011, pp. 1–6.

[4] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] P. Geurts, D. Ernst, and L. Wehenkel, “Extremely randomized trees,” *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.

[6] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[8] E. Jonas *et al.*, “Cloud programming simplified: A Berkeley view on serverless computing,” Univ. California, Berkeley, Tech. Rep. UCB/EECS-2019-3, 2019.

[9] OASIS, “MQTT Version 3.1.1,” OASIS Standard, Oct. 2014.

[10] Amazon Web Services, “AWS IoT Core Developer Guide.” [Online]. Available: https://docs.aws.amazon.com/iot/latest/developerguide/

[11] H. Butler, M. Daly, A. Doyle, S. Gillies, S. Hagen, and T. Schaub, “The GeoJSON format,” IETF RFC 7946, Aug. 2016.

[12] V. Agafonkin, “Leaflet: An open-source JavaScript library for mobile-friendly interactive maps.” [Online]. Available: https://leafletjs.com/

[13] G. Niemeyer, “Geohash,” 2008. [Online]. Available: http://geohash.org/

[14] ISO 8608:2016, *Mechanical vibration — Road surface profiles — Reporting of measured data*. International Organization for Standardization, 2016.

[15] Espressif Systems, “ESP-IDF Programming Guide.” [Online]. Available: https://docs.espressif.com/projects/esp-idf/

[16] Amazon Web Services, “AWS Serverless Application Model Developer Guide.” [Online]. Available: https://docs.aws.amazon.com/serverless-application-model/

APPENDIX A: IOT POLICY AND IOT RULE

```
{"Version": "2012-10-17", "Statement": [
  {"Effect": "Allow", "Action": "iot:Connect",
   "Resource": "arn:aws:iot:ap-south-1:<account>:client/*"},
  {"Effect": "Allow", "Action": "iot:Publish",
   "Resource": "arn:aws:iot:ap-south-1:<account>:topic/pothole/alerts"}]}
```

```
SELECT *, timestamp() AS received_ms, clientid() AS clientid FROM
'pothole/alerts'
```

APPENDIX B: REPOSITORY STRUCTURE

```
components/        shared ESP-IDF components: accel, neo7m_gps, road_event,
pothole_model, board
esp32_collector/   firmware: stream raw data for training
esp32_inference/   firmware: offline bench test
esp32_iot/         firmware: detection + GPS + AWS IoT MQTT
ml/                collect, review, dataset, simulate, augment, train, tests
cloud/             SAM template, Lambda code, map.html, provision_device.sh,
tests
model/report.md    latest training report
docs/              documentation, figures, presentation, reports
```